

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
(Universidad del Perú, DECANA DE AMÉRICA)
FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA



DOCENTE:

Villafuerte Barreto, Hernan Oswaldo

CURSO:

Circuitos Digitales

INTEGRANTES:

Melendez Romero Jose Francesco

Sabrera Ortiz, Angie Roxana

Berrocal Casanca Esaú Andree

LIMA - PERÚ

2026

TÍTULO: Implementación de un sistema digital de validación de integridad por paridad longitudinal con banco de registros direccionables.

1. Introducción

En la transmisión de datos digitales es común que se presenten errores debido al ruido o interferencias en el canal de comunicación. Estos errores pueden alterar uno o más bits de la información enviada, ocasionando que los datos lleguen de manera incorrecta al receptor. Por ello, es importante implementar mecanismos que permitan verificar la integridad de la información y detectar posibles fallas durante la transmisión.

El presente proyecto consiste en el diseño e implementación de un sistema digital capaz de verificar la integridad de tramas de 8 bits mediante el uso de checksum basado en operaciones XOR. El sistema recibe una trama de datos junto con su checksum correspondiente, realiza la verificación y determina si la información fue transmitida correctamente. Cuando la trama es válida, esta se almacena en un banco de registros direccionables; en caso contrario, el sistema activa una señal de error para indicar la inconsistencia detectada. Además, el funcionamiento general del sistema es controlado mediante una máquina de estados finitos (FSM), junto con un contador y un decodificador encargados del direccionamiento de memoria.

El **objetivo principal** de este trabajo es diseñar e implementar un sistema digital que permita detectar errores en la transmisión de datos y almacenar las tramas válidas utilizando lógica combinacional y secuencial. Asimismo, se busca aplicar de manera práctica distintos conceptos desarrollados en el curso, como máquinas de estados, flip-flops, decodificadores, memorias y circuitos lógicos, utilizando herramientas de simulación digital para comprobar el correcto funcionamiento del sistema.

2. Descripción del sistema

➤ Funcionamiento del sistema

El sistema se encarga de verificar la integridad de tramas de datos de 8 bits utilizando el método de checksum XOR. Primero, el sistema recibe la trama de datos y el checksum asociado. Luego, mediante operaciones XOR, calcula el checksum esperado y lo compara con el valor recibido.

- Si ambos valores coinciden:
 - La trama se considera válida.
 - La información se almacena en un banco de registros.
 - El sistema avanza a la siguiente posición de memoria.
- Si los valores no coinciden:

- Se detecta un error en la transmisión.
- Se activa una señal de error mediante un LED o indicador.

Todo el proceso es controlado mediante una máquina de estados finitos (FSM), la cual coordina la recepción, verificación y almacenamiento de datos.

➤ Entradas y salidas

Entradas:

- DATA_IN [7:0]: trama de datos de 8 bits.
- CHECKSUM_IN: checksum recibido junto con la trama.
- CLOCK: señal de reloj para sincronizar el sistema.
- RESET: reinicia el sistema.
- ENABLE / START: inicia el proceso de verificación.

Salidas:

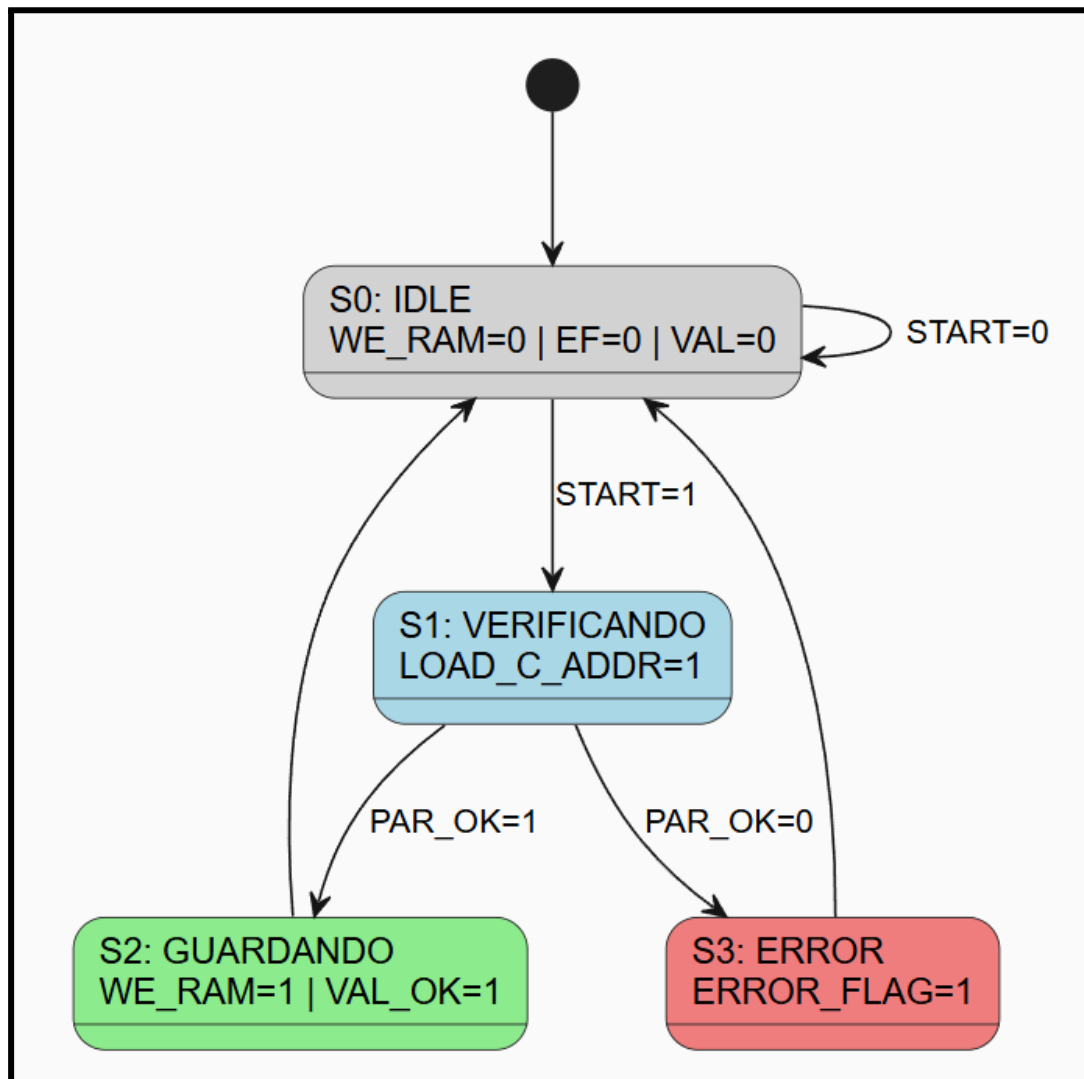
- ERROR_FLAG: indica si existe un error en la trama.
- DATA_OUT: salida de los datos almacenados.
- ADDRESS_OUT: dirección actual del banco de registros.
- LEDs indicadores: muestran estados del sistema o presencia de errores.

➤ Componentes del sistema

1. **Módulo XOR:** Encargado de calcular el checksum esperado mediante operaciones XOR bit a bit.
2. **Comparador:** Compara el checksum calculado con el checksum recibido para determinar si existe error.
3. **FSM (Máquina de Estados Finitos):** Controla las etapas del sistema:
 - Recepción de datos
 - Verificación
 - Almacenamiento
 - Señalización de error
4. **Banco de registros:** Memoria donde se almacenan las tramas válidas.
5. **Contador de 2 bits:** Genera la dirección de almacenamiento para seleccionar la posición del registro.
6. **Decodificador:** Selecciona qué registro será habilitado para guardar la información.
7. **Arduino:** Se utilizará un Arduino como plataforma de implementación y control del sistema debido a su menor costo y facilidad de uso. Este permitirá realizar pruebas, controlar entradas y salidas, además de facilitar la simulación del funcionamiento general del proyecto.

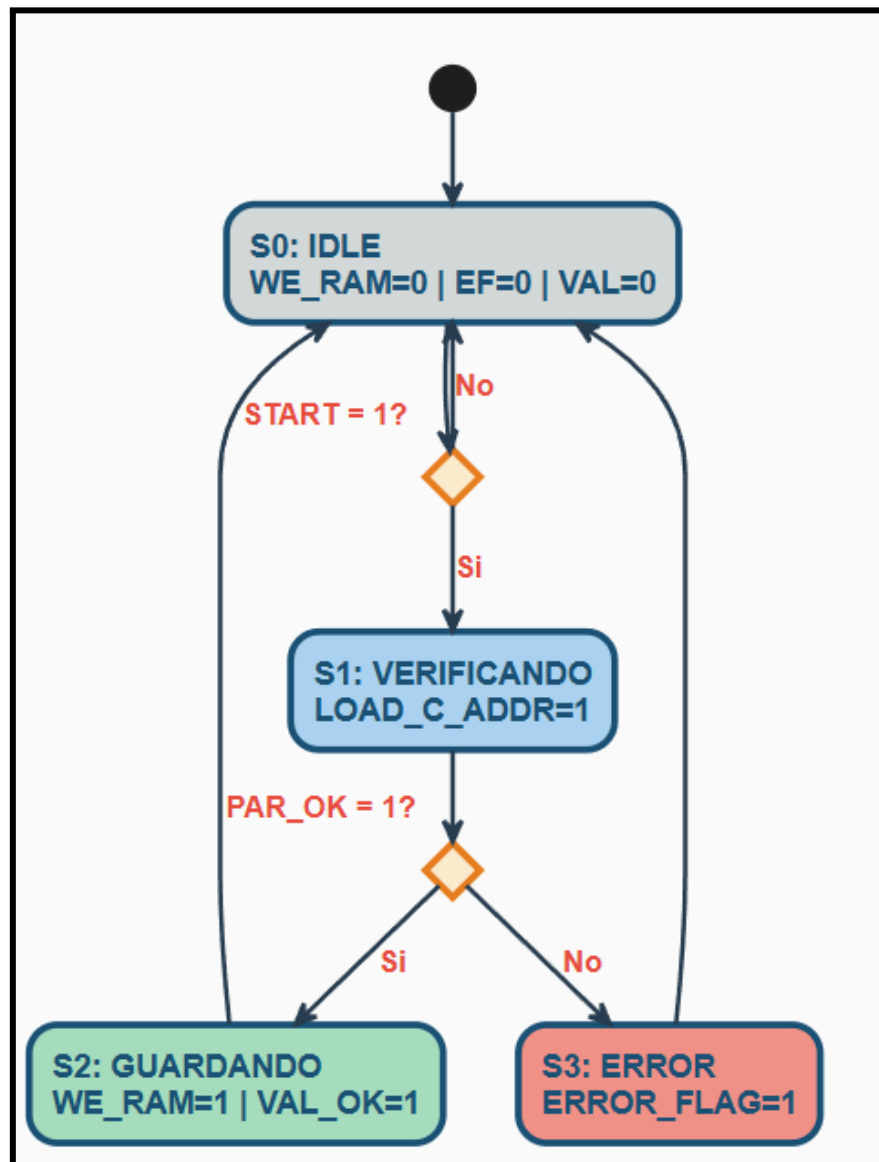
3. Diseño del sistema

3.1 Diagrama FSM



- El sistema arranca siempre en S0 (IDLE), que es básicamente el estado de espera. Ahí no pasa nada, todas las señales están en cero y el sistema simplemente espera que alguien active la señal START.
- Cuando START llega, en el siguiente flanco de reloj el sistema pasa a S1 (VERIFICANDO). En ese momento se activa LOAD_C_ADDR y el árbol XOR ya está calculando la paridad de la trama recibida. Dependiendo de ese resultado hay dos caminos: si la paridad es correcta (PAR_OK=1) el sistema va a S2 (GUARDANDO), donde se activa WE_RAM para escribir la trama en memoria y VAL_OK para indicar que todo salió bien. Si la paridad falla (PAR_OK=0), el sistema cae en S3 (ERROR) y se activa ERROR_FLAG. En ambos casos, el sistema regresa solo a IDLE para esperar la siguiente trama.

3.2 Diagrama ASM



- El diagrama ASM muestra el mismo comportamiento pero de forma más detallada, dejando claro en qué momento se evalúa cada condición y qué salidas se activan en cada paso.
- El flujo empieza en S0 (IDLE). El primer rombo pregunta si START vale 1. Mientras no llegue esa señal, el sistema se queda dando vueltas en IDLE. Cuando START se activa, se entra al bloque S1 (VERIFICANDO) con LOAD_C_ADDR=1, y justo después viene el segundo rombo que evalúa PAR_OK. Si la paridad es correcta el flujo va hacia la izquierda, entra a S2 (GUARDANDO) con WE_RAM=1 y VAL_OK=1, y luego regresa a IDLE. Si hay error, el flujo va hacia la derecha, entra a S3 (ERROR) con ERROR_FLAG=1, y también regresa a IDLE. El ciclo se repite para cada trama que llegue al sistema.

3.3 Tablas de verdad

- TABLA 1 – Decodificador RAM (2 bits → 4 líneas)

A1	A0	SEL[0]	SEL[1]	SEL[2]	SEL[3]	Registro
0	0	1	0	0	0	ram[0]
0	1	0	1	0	0	ram[1]
1	0	0	0	1	0	ram[2]
1	1	0	0	0	1	ram[3]

- TABLA 2 – Verificador XOR (casos representativos)

$XOR_calc = D7 \oplus D6 \oplus D5 \oplus D4 \oplus D3 \oplus D2 \oplus D1 \oplus D0$. $PAR_OK = 1$ si $XOR_calc =$
 PARIDAD_ENTRADA:

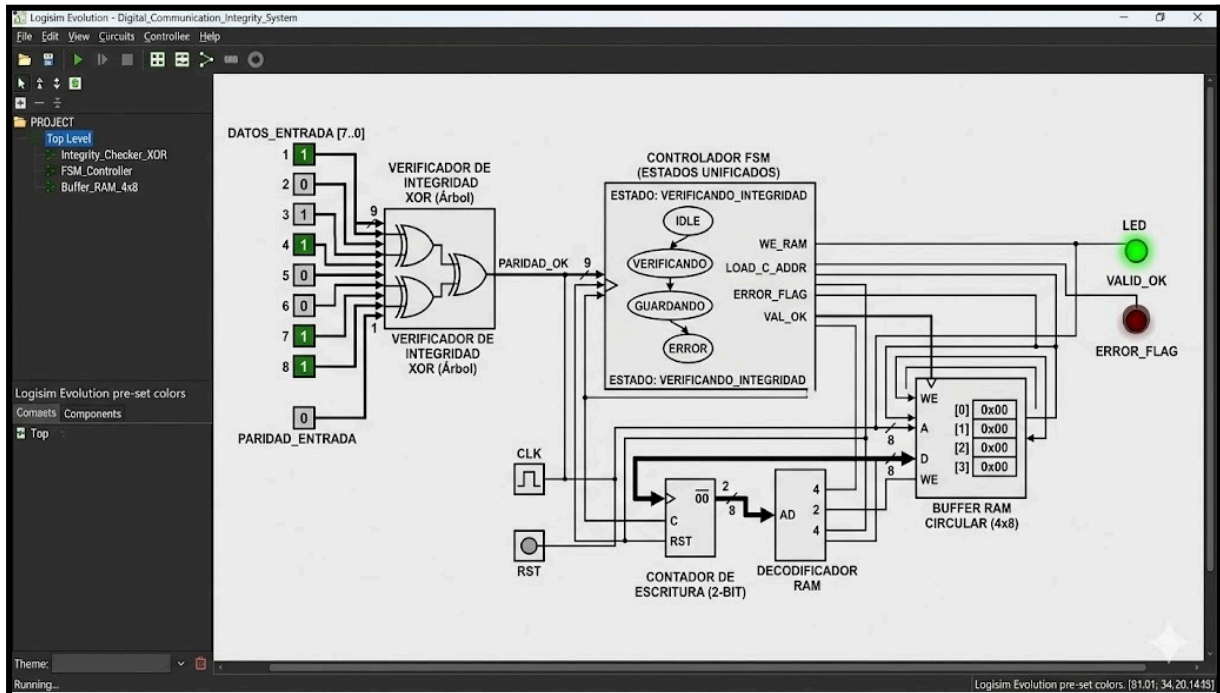
D[7:0] (ejemplo)	XOR_calc	PARIDAD_ENTRADA	PAR_OK	Resultado
0000 0000	0	0	1	valido
1000 0001	0	0	1	valido
1010 1001	1	1	1	valido
1010 1000	1	0	0	error
0110 0011	0	1	0	error

- TABLA 3 – Transición de estados FSM

Estado actual	START	PAR_OK	Estado siguiente	WE_RAM	LOAD_C_ADDR	ERROR_FLAG	VAL_OK
IDLE	0	X	IDLE	0	0	0	0
IDLE	1	X	VERIFICANDO	0	0	0	0
VERIFICANDO	X	1	GUARDANDO	0	1	0	0
VERIFICANDO	X	0	ERROR	0	1	0	0
GUARDANDO	X	X	IDLE	1	0	0	1

GUARDANDO	X	X	IDLE	0	0	1	IDLE
-----------	---	---	------	---	---	---	------

3.4 Diseño lógico / circuito



- El circuito fue diseñado e implementado en Logisim Evolution, organizando el sistema en bloques que trabajan en conjunto para verificar y almacenar las tramas de datos.
- La entrada principal son los 8 bits de datos (DATOS_ENTRADA[7..0]) junto con el bit de paridad (PARIDAD_ENTRADA). Estos ingresan al bloque verificador XOR, que usa un árbol de compuertas para calcular el XOR de todos los bits. Si el resultado coincide con la paridad recibida, se genera la señal PARIDAD_OK que le indica a la FSM que el dato es válido.
- La FSM es el núcleo del sistema. Coordina todo el proceso: desde la recepción del dato hasta su almacenamiento o la activación de la señal de error. Dependiendo del estado en que se encuentre, activa las señales de control necesarias para los demás bloques.
- Para el direccionamiento de la memoria se usa un contador de 2 bits que lleva la cuenta de cuántas tramas se han guardado. Su salida pasa por un decodificador que activa únicamente el registro correcto dentro del Buffer RAM de 4×8 bits, donde se almacenan las tramas válidas de forma secuencial.

- Por último, el sistema cuenta con dos LEDs indicadores: uno verde que confirma cuando una trama fue almacenada correctamente, y uno rojo que se enciende cuando se detecta un error de paridad en la trama recibida.

4. Código HDL o pseudocódigo

5. Simulación funcional

5.1 Plan de simulación

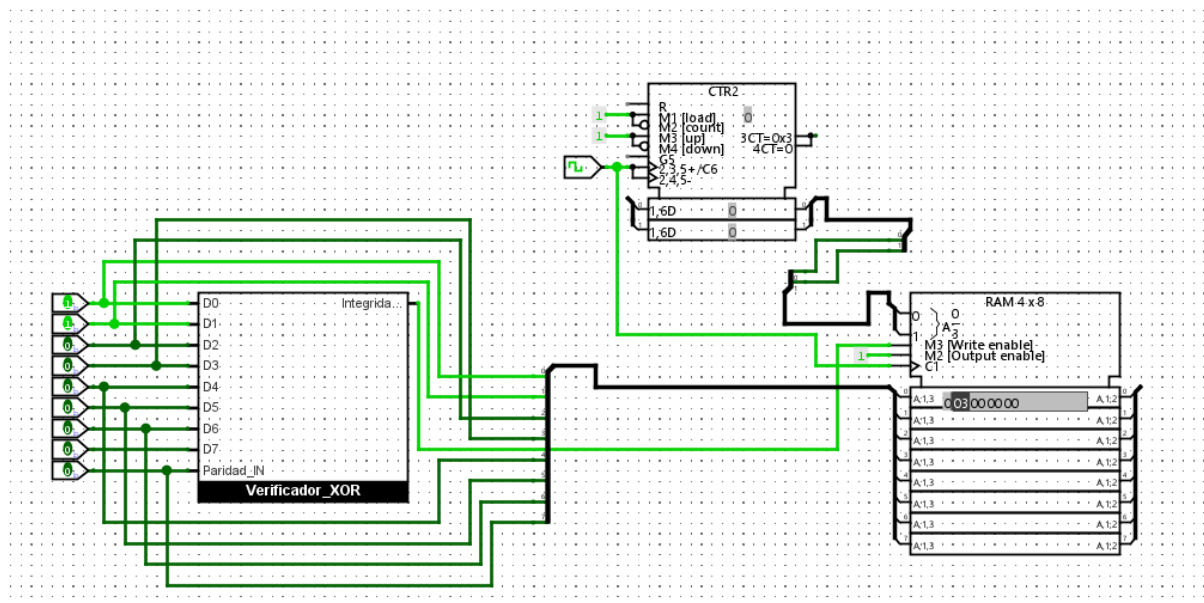
- La simulación se realizó en Logisim Evolution con el objetivo de verificar el correcto funcionamiento de cada bloque del sistema. Se diseñaron pruebas para cubrir los tres caminos posibles del sistema: trama válida, trama con error, y reset del sistema.
- Las pruebas se organizaron así:
 - Verificar que el sistema arranca en IDLE y espera la señal START
 - Comprobar que el verificador XOR detecta correctamente tramas válidas
 - Comprobar que el verificador XOR detecta correctamente tramas con error
 - Verificar que las tramas válidas se guardan en el registro correcto del Buffer RAM
 - Verificar que el contador de dirección avanza correctamente tras cada escritura
 - Comprobar que el LED verde (VAL_OK) y el LED rojo (ERROR_FLAG) responden correctamente
 - Verificar el funcionamiento del RST

5.2 Casos de prueba

Caso	DATA_IN	PARIDAD_IN	PAR_OK esperado	Estado esperado	Resultado
Trama válida, paridad correcta	10000001	0	1	GUARDANDO	✓
Trama válida, paridad correcta	10101001	1	1	GUARDANDO	✓
Trama con error, paridad incorrecta	10101000	0	0	ERROR	✓

Trama todo ceros, paridad 0	00000000	0	1	GUARDANDO	✓
Trama todo unos, paridad 0	11111111	0	1	GUARDANDO	✓
Reset en cualquier estado	cualquiera	cualquiera	X	IDLE	✓
4 tramas válidas consecutivas	varias	correctas	1	RAM[0]→RAM[3]	✓

5.3 Resultados



En esta sección se detalla el análisis de la simulación funcional realizada en **Logisim Evolution**, demostrando la capacidad del sistema para procesar y validar información en tiempo real. Este resultado valida experimentalmente el diseño lógico propuesto. El sistema no solo detecta errores de transmisión mediante el árbol XOR, sino que garantiza que solo las tramas que cumplen con el protocolo de integridad ocupen un espacio en el banco de registros, optimizando así el uso de la memoria circular.

6. Demostración del funcionamiento

- ¿Cómo funciona el sistema completo?

El sistema arranca siempre en estado IDLE, con todos los LEDs apagados y el contador de dirección en cero. Cuando llega una trama de 8 bits junto con su bit de paridad, el usuario activa la señal START y el sistema entra al estado VERIFICANDO.

En ese momento, el árbol XOR calcula la paridad de los 8 bits recibidos y la compara con el bit de paridad que vino junto con la trama. Si ambos coinciden, el sistema sabe que la

información llegó sin errores y pasa al estado GUARDANDO. Si no coinciden, algo se corrompió en la transmisión y el sistema pasa al estado ERROR.

Cuando todo está bien y el sistema llega a GUARDANDO, el LED verde se enciende por un momento confirmando que la trama fue aceptada. Cuando hay error, es el LED rojo el que se enciende indicando que esa trama fue descartada. En ambos casos el sistema regresa solo a IDLE para esperar la siguiente trama.

- ¿Cómo trabaja el decodificador?

El decodificador es el encargado de decidir en cuál de los 4 registros del Buffer RAM se guarda cada trama válida. Recibe 2 bits del contador y los convierte en una sola línea activa de las 4 posibles. Por ejemplo, si el contador vale 00, activa únicamente la línea del registro 0. Si vale 01, activa el registro 1, y así sucesivamente.

Esto garantiza que nunca se escriba en dos registros al mismo tiempo, y que cada trama válida ocupe su propio espacio en la memoria.

- ¿Cómo accede a la memoria?

Cada vez que una trama válida se guarda exitosamente, el contador de 2 bits se incrementa en uno, apuntando al siguiente registro disponible. Como el contador es de 2 bits, cuando llega al registro 3 (dirección 11) vuelve automáticamente al registro 0, funcionando como un buffer circular.

La escritura solo ocurre cuando la FSM activa la señal WE_RAM en el estado GUARDANDO. Fuera de ese estado, la memoria no se modifica, así que los datos guardados permanecen intactos mientras el sistema sigue verificando nuevas tramas.

7. Conclusiones

Este proyecto nos ayudó a entender cosas que en clase se ven muy abstractas. Por ejemplo, la FSM en el pizarrón parece simple, pero cuando la tienes que implementar en Logisim y conectar con los demás bloques te das cuenta de que hay que pensar bien cada señal y cada transición, porque un error en un estado arruina todo lo demás.

Lo del árbol XOR también fue interesante. Al principio no teníamos muy claro cómo calcular la paridad en hardware sin usar una sola compuerta enorme, y resulta que dividirlo en niveles como un árbol hace que sea mucho más ordenado y fácil de escalar.

El decodificador fue el bloque que más nos costó entender al principio, pero una vez que vimos que básicamente es un "selector" que dice "escribe aquí y no en los demás", todo tuvo

más sentido. Lo mismo con el contador de 2 bits — es pequeño pero sin él no hay forma de llevar la dirección de la memoria.

La simulación funcionó bien en todos los casos que probamos. Las tramas válidas se guardaron donde debían y el LED de error se activó cuando la paridad no coincidía, que era exactamente lo que buscábamos.

El mayor problema que tuvimos fue no contar con la FPGA, lo que nos obligó a buscar alternativas. Al final eso también nos enseñó algo: que el mismo diseño puede adaptarse a diferentes plataformas sin perder su lógica de fondo.

8. Anexos

- Código completo

```
// -----  
  
// MÓDULO 1: Verificador de paridad XOR en árbol  
  
//  
// Toma los 8 bits de la trama y calcula su paridad usando  
// compuertas XOR organizadas en árbol. Luego compara el  
// resultado con la paridad que llegó junto con la trama.  
// Si coinciden, PAR_OK = 1. Si no, hay un error.  
// -----  
  
module verificador_xor (  
    input [7:0] datos,    // trama de 8 bits de entrada  
    input    paridad_rx, // paridad recibida con la trama  
    output    par_ok     // 1 = trama válida, 0 = error  
);  
  
// Primer nivel del árbol: XOR entre pares de bits  
wire x1 = datos[0] ^ datos[1];  
wire x2 = datos[2] ^ datos[3];
```

```
wire x3 = datos[4] ^ datos[5];
```

```
wire x4 = datos[6] ^ datos[7];
```

```
// Segundo nivel: combinamos los resultados anteriores
```

```
wire x5 = x1 ^ x2;
```

```
wire x6 = x3 ^ x4;
```

```
// Tercer nivel: resultado final del XOR de los 8 bits
```

```
wire xor_calc = x5 ^ x6;
```

```
// Comparamos con la paridad recibida
```

```
assign par_ok = (xor_calc == paridad_rx) ? 1'b1 : 1'b0;
```

```
endmodule
```

```
// -----
```

```
// MÓDULO 2: Contador de 2 bits
```

```
//
```

```
// Lleva la cuenta de en qué registro del Buffer RAM
```

```
// se va a guardar la siguiente trama válida.
```

```
// Cuando llega a 3, vuelve a 0 automáticamente
```

```
// porque es un contador de 2 bits (0, 1, 2, 3, 0, 1...)
```

```
// -----
```

```

module contador_2bit (
    input      clk,
    input      rst,
    input      incrementar, // sube solo cuando se guarda una trama
    output reg [1:0] direccion // dirección actual (0 a 3)
);

always @(posedge clk or posedge rst) begin
    if (rst)
        direccion <= 2'b00; // reset: volvemos al registro 0
    else if (incrementar)
        direccion <= direccion + 1; // avanzamos al siguiente registro
    end

endmodule

```

```

// -----
// MÓDULO 3: Decodificador 2 a 4
//
// Convierte la dirección de 2 bits del contador en 4 líneas
// de selección. Solo una línea puede estar activa a la vez,
// lo que garantiza que solo se escribe en un registro.
// Si WE (write enable) está inactivo, ninguna línea se activa.
// -----

```

```

module decodificador_2a4 (
    input    [1:0] dir, // dirección de 2 bits del contador
    input     we, // write enable: viene de la FSM
    output reg [3:0] sel // 4 líneas de selección (una por registro)
);

always @(*) begin
    if (we) begin
        // Activamos solo la línea que corresponde
        case (dir)
            2'b00: sel = 4'b0001; // escribe en RAM[0]
            2'b01: sel = 4'b0010; // escribe en RAM[1]
            2'b10: sel = 4'b0100; // escribe en RAM[2]
            2'b11: sel = 4'b1000; // escribe en RAM[3]
            default: sel = 4'b0000;
        endcase
    end else
        sel = 4'b0000; // sin escritura: todas las líneas inactivas
    end

endmodule

```

```
// -----
```

```
// MÓDULO 4: Buffer RAM circular 4x8
```

```

//
// Almacena hasta 4 tramas válidas de 8 bits cada una.
// La escritura se controla con las líneas sel del decodificador.
// Los datos se mantienen hasta que se sobrescriben.
// -----
module buffer_ram (
    input      clk,
    input  [3:0] sel, // líneas de selección del decodificador
    input  [7:0] datos, // trama a guardar
    output reg [7:0] ram0, // contenido del registro 0
    output reg [7:0] ram1, // contenido del registro 1
    output reg [7:0] ram2, // contenido del registro 2
    output reg [7:0] ram3 // contenido del registro 3
);
    always @(posedge clk) begin
        // Cada registro solo se actualiza si su línea está activa
        if (sel[0]) ram0 <= datos;
        if (sel[1]) ram1 <= datos;
        if (sel[2]) ram2 <= datos;
        if (sel[3]) ram3 <= datos;
    end

endmodule

```

```

// -----

// MÓDULO 5: Controlador FSM

//

// Es el núcleo del sistema. Decide qué hace el sistema
// en cada momento según el estado en que se encuentra
// y las señales que recibe del exterior.

//

// Estados:

// IDLE    → espera que llegue una trama
// VERIFICANDO → carga la trama y evalúa la paridad
// GUARDANDO → trama válida, escribe en RAM
// ERROR    → paridad incorrecta, activa señal de error

// -----

module fsm_controller (
    input  clk,
    input  rst,
    input  start,    // señal de inicio
    input  par_ok,    // resultado del verificador XOR
    output reg we_ram,    // habilita escritura en RAM
    output reg load_c_addr, // habilita carga de dirección
    output reg error_flag, // indica error de paridad
    output reg val_ok     // indica trama guardada correctamente
);

```



```

// Definimos los 4 estados con nombres descriptivos

parameter IDLE      = 2'b00;

parameter VERIFICANDO = 2'b01;

parameter GUARDANDO  = 2'b10;

parameter ERROR      = 2'b11;


reg [1:0] estado_actual;

reg [1:0] estado_siguiente;


// Registro secuencial: actualiza el estado en cada flanco de reloj
always @(posedge clk or posedge rst) begin

    if (rst)

        estado_actual <= IDLE; // reset: volvemos al inicio

    else

        estado_actual <= estado_siguiente;

end


// Lógica combinacional: calcula el próximo estado
always @(*) begin

    case (estado_actual)

        IDLE:

            // Esperamos START para empezar a verificar

            if (start) estado_siguiente = VERIFICANDO;

            else      estado_siguiente = IDLE;

    end

```

VERIFICANDO:

// Según la paridad, decidimos si guardar o reportar error

if (par_ok) estado_siguiente = GUARDANDO;

else estado_siguiente = ERROR;

GUARDANDO:

// Después de guardar, volvemos a esperar

estado_siguiente = IDLE;

ERROR:

// Después de reportar el error, volvemos a esperar

estado_siguiente = IDLE;

default:

estado_siguiente = IDLE;

endcase

end

// Lógica de salidas: Máquina de Moore

// Las salidas dependen únicamente del estado actual

always @(*) begin

// Por defecto todo apagado

we_ram = 1'b0;

```

load_c_addr = 1'b0;

error_flag = 1'b0;

val_ok     = 1'b0;


case (estado_actual)
    VERIFICANDO: begin
        load_c_addr = 1'b1; // cargando y verificando la trama
    end

    GUARDANDO: begin
        we_ram = 1'b1;    // escribir en RAM
        val_ok = 1'b1;    // encender LED verde
    end

    ERROR: begin
        error_flag = 1'b1; // encender LED rojo
    end

endcase

end

endmodule


// -----
// MÓDULO TOP: Sistema completo
//

```

```

// Conecta todos los módulos anteriores entre sí.

// Este es el circuito completo listo para sintetizar
// en una FPGA o simular en Logisim/Quartus.

// -----

module top_level (

    input    clk,

    input    rst,

    input    start,

    input  [7:0] datos_entrada,  // trama de 8 bits

    input    paridad_entrada, // bit de paridad recibido

    output   val_ok,           // LED verde

    output   error_flag       // LED rojo

);

// Señales internas que conectan los módulos

wire    par_ok;

wire    we_ram;

wire    load_c_addr;

wire [1:0] direccion;

wire [3:0] sel;

wire [7:0] ram0, ram1, ram2, ram3;


// U1: Verificador XOR

verificador_xor U1 (

    .datos    (datos_entrada),

```

```
.paridad_rx (paridad_entrada),  
.par_ok    (par_ok)  
);
```

```
// U2: Controlador FSM
```

```
fsm_controller U2 (  
    .clk      (clk),  
    .rst      (rst),  
    .start    (start),  
    .par_ok   (par_ok),  
    .we_ram   (we_ram),  
    .load_c_addr(load_c_addr),  
    .error_flag (error_flag),  
    .val_ok   (val_ok)  
);
```

```
// U3: Contador de 2 bits
```

```
contador_2bit U3 (  
    .clk      (clk),  
    .rst      (rst),  
    .incrementar(we_ram),  
    .direccion (direccion)  
);
```

```
// U4: Decodificador 2 a 4
```

```
decodificador_2a4 U4 (
```

```
    .dir (direccion),
```

```
    .we (we_ram),
```

```
    .sel (sel)
```

```
);
```

```
// U5: Buffer RAM 4x8
```

```
buffer_ram U5 (
```

```
    .clk (clk),
```

```
    .sel (sel),
```

```
    .datos(datos_entrada),
```

```
    .ram0 (ram0),
```

```
    .ram1 (ram1),
```

```
    .ram2 (ram2),
```

```
    .ram3 (ram3)
```

```
);
```

```
endmodule
```

- Capturas extra
- Diagramas grandes